Agentic AI PMS: Specification & Workflow Architecture

Version: 4.0 (Vertex AI Native)
Status: Draft / Active

1. System Overview

"Visual Management, Agentic Execution"

The core philosophy of this system is **Hybrid Automation**.

- **The "Agents" (AI/Bots):** Powered by **Vertex AI Reasoning Engine**. They execute tasks (data entry, syncing) in the background.
- **The "Dashboard" (Visual):** A control tower for the Human Operator. It displays the status of the Agents. The human only intervenes when an Agent reports an "Anomaly" (Manage by Exception).

---

2. Technology Stack (The "Build")

A **Fully Managed** stack. We remove self-hosted containers (like Phoenix) in favor of Google's managed AI runtime.

2.1 Core Infrastructure

- **Orchestrator: Google Cloud Tasks**.
  - *Role:* The "Scheduler". Zero-cost queue for future tasks (e.g., "Send Guide in 3 days").
- **Compute: Google Cloud Run**.
  - *Role:* Hosting the **Next.js Frontend**.
- **AI Comput: Vertex AI Reasoning Engine**.
  - *Role:* Hosting the **Agents**. It is a managed service that wraps your Python code and LangChain agents, handling auto-scaling and logging automatically.
- **Data Layer: Google Cloud Firestore**.
  - *Role:* Single Source of Truth for Reservations, Rooms, Availability, and History.

2.2 Frontend (The "Control Tower")

- **Framework: Next.js (React)**.
- **Hosting:** Firebase Hosting (rewrites to Cloud Run).
- **Design:** Bespoke "Flight Control" Dashboard.
- **Data Fetching:** Real-time Firestore listeners.

2.3AI & Observability (The Big Change)

- ~~Arize Phoenix~~ -> **Vertex AI Tracing (Cloud Logging)**.
  - **Simplification:** No need to deploy/manage a Phoenix container.
  - **Functionality:** Vertex AI automatically logs agent "thoughts" (inputs, chain-of-thought, tool outputs) to Google Cloud Logging.
  - **Dashboard:** You view traces directly in the Google Cloud Console "Trace" view if you need to debug.

---

3. Cost & Benefit Analysis (V3 vs. V4)

3.1 Simplification Benefits

1. **Zero DevOps for AI:** You don't manage Docker containers for the agents or the observer. Vertex handles the runtime.
2. **Unified Security:** Agents run with Google Service Accounts natively. No API keys floating around for third-party trackers.
3. **Future Proofing:** Vertex AI Reasoning Engine is Google's flagship for agents; you get new models (Gemini 1.5 Pro/Flash) and features instantly.

3.2 Cost Estimates (Low Volume < 10k bookings)

- **Compute (Cloud Run):** ~$5-10/mo (Frontend mainly idle).

- **Database (Firestore):** ~$0-5/mo (Free tier handles 50k reads/day).
- **Task Queue:** $0 (Free tier 1M tasks/mo).
- **AI (Vertex AI):**
  - *Gemini 1.5 Flash:* $0.35 / 1M input tokens. (Extremely cheap).
  - *Reasoning Engine Runtime:* ~$15-30/mo (Estimated for "always available" endpoints, but scales to zero).
  - *Savings:* You save ~$50/mo by NOT running a dedicated Arize Phoenix server.

Total Est. Monthly: < $50 (mostly the AI Runtime + Gemini usage).

---

4. The Triggers & Workflows

Phase 1: The Booking Event

Trigger: New Reservation Webhook (e.g., from Airbnb/Booking.com)

Agent Task 1: "The Gatekeeper" (Auto-Responder)

- **Runtime:** Vertex AI Agent.
- **Action:** Query Firestore `rooms` & `availability`.
- **Action:** Create Document in `reservations` (Status: `NEW`).

Agent Task 2: "The Integrator" (Opera Sync)

- **Runtime:** Cloud Run Job (Headless Browsers are heavy, better as a Job than a Reasoning Engine function).
- **Action:** Log into Opera.
- **Visual Proof:** Capture Screen Print -> **GCS**.
- **Output:** Update Firestore with `opera_conf`.

Agent Task 3: "The Concierge" (Initial Confirmation)

- **Runtime:** Vertex AI Agent.
- **Trigger:** Firestore `onUpdate` -> Pub/Sub -> Agent.
- **Action:** Send Multi-channel Confirmation.
- **Scheduling: Enqueue Cloud Task** -> Target: "The Guide" (T-3).

Phase 2: The Pre-Arrival Countdown (Tasks 4 & 5)

- **Mechanism:** Cloud Task wakes up a Vertex AI Agent at the specific time to send the Guide or Check-in Nudge.

---

5. The Visual Dashboard (UI Specification)

Visual Metaphor: Kanban or "Flight Control" Board
Rows: Reservations (from Firestore). **Cols:** Workflow Steps.

Guest Name	Arr Date	Step 1: OTA	Step 2: Opera	Step 3: Confirm	Step 4: Guide (T-3)	Step 5: Check-in (T-1)	Status
John Doe	Jan 20	✅ Accepted	✅ Conf: #12345 (View Image)	✅ WhatsApp Sent	⏳ Pending (T-3)	⏳ Pending (T-1)	🟢 On Track
Jane Smith	Jan 21	✅ Accepted	❌ ERROR: Rate Mismatch	⏸️ Held	⏸️ Held	⏸️ Held	🔴 ACTION REQ

Human Interactions (Audit Log):

- **Action:** James updates Check-out date.
- **Effect:** Dashboard writes to `reservations/{id}/audit_logs` (Firestore).

---

6. Simulation Data (Scenario: Reservation Capture)

6.1 Firestore Data Model (NoSQL)

Collection: `reservations`
Document ID: `res_WFV_1029`

```
{
  "platform_id": "HM-QZ9921",
  "guest": {
    "name": "Michael",
    "phone": "+15550199"
  },
  "dates": {
    "check_in": "2025-11-10",
    "check_out": "2025-11-15"
  },
  "property": {
    "code": "WFV",
    "room": "WFV-E003"
  },
  "workflow": {
    "status": "SYNCED",
    "opera_conf": "8829103",
    "screenshot_url": "gs://bucket/reservations/HM-QZ9921_opera.png",
    "confirmation_sent": true,
    "guide_sent": false
  },
  "meta_payload": { ... } // Full Airbnb JSON
}
```

Subcollection: `reservations/{res_id}/audit_logs` (History)

```
{
  "log_id": "log_88291",
  "timestamp": "2025-12-11T10:05:00Z",
  "actor": "James (Human Operator)",
  "action": "UPDATE_CHECKOUT",
  "changes": {
    "field": "dates.check_out",
    "old_value": "2025-02-11",
    "new_value": "2025-02-15"
  }
}
```

Subcollection: `reservations/{res_id}/ledger` (Financials)

```
{
  "tx_id": "tx_9921",
  "timestamp": "2025-11-10T08:00:00Z",
  "type": "PAYOUT_RECEIVED",
  "amount_cents": 125000,
  "currency": "USD",
  "source": "Airbnb",
  "status": "CLEARED"
}
```